

NAME: Shaikh Areeba Mohammed Ismail

CLASS/BATCH: TE-B2

ROLL NO: 46

SUBJECT: Artificial Intelligence

GROUP: A

ASSIGNMENT NO: 01

Problem Statement: Implement depth first search algorithm and Breadth First Search algorithm, use an undirected graph and develop a recursive algorithm for searching all the vertices of a Graph or tree data structure.

Step 1: Define an Undirected Graph using Adjacency List

Each key represents a node.

Each value is a list of adjacent (connected) nodes.

```
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A', 'F'],
    'D': ['B'],
    'E': ['B', 'F'],
    'F': ['C', 'E']
}

print ("Undirected Graph Representation:")
for node in graph:
    print (node, "->", graph[node])
```

OUTPUT:

Undirected Graph Representation:

```
A -> ['B', 'C']
B -> ['A', 'D', 'E']
C -> ['A', 'F']
D -> ['B']
E -> ['B', 'F']
F -> ['C', 'E']
```

Step 2: Depth First Search (DFS) - Recursive Implementation

```
def dfs(graph, node, visited=None):
    """
    Recursive DFS function
    graph: adjacency list representation
    node : starting node
    visited: set to keep track of visited nodes
    """

    # Create visited set if it does not exist
    if visited is None:
        visited = set ()

    # Mark current node as visited
    visited.add(node)
    print (node, end=' ')

    # Visit all unvisited neighbors recursively
    for neighbor in graph[node]:
        if neighbor not in visited:
            dfs(graph, neighbor, visited)

    return visited

print ("\nDFS Traversal starting from 'A':")
dfs (graph, 'A')
```

OUTPUT:

```
DFS Traversal starting from 'A':
A B D E F C
{'A', 'B', 'C', 'D', 'E', 'F'}
```

Step 3: Breadth First Search (BFS) Implementation

```
from collections import deque

def bfs(graph, start_node):
    """
    BFS function using queue
    graph      : adjacency list
    start_node: node to start traversal
    """

    visited = set ()           # Track visited nodes
    queue = deque ()          # Queue for BFS

    visited.add(start_node)    # Mark start node visited
    queue.append(start_node)   # Enqueue start node

    while queue:
        node = queue.popleft() # Remove from queue
        print (node, end=' ')

        # Add unvisited neighbors to queue
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)

    return visited

print("\nBFS Traversal starting from 'A':")
bfs(graph, 'A')
```

OUTPUT:

```
BFS Traversal starting from 'A':
A B C D E F
{'A', 'B', 'C', 'D', 'E', 'F'}
```